

PMTBench

PMTBench is an open-source framework that helps researchers work with datasets and models for Predictive Mutation Testing (PMT). It is designed to improve how PMT datasets are documented, shared, and reused, and to support the experimentation with PMT models, making it easier to compare results and reproduce PMT studies.

PMTBench has been designed to be as general and flexible as possible, so it can be used in the scenarios commonly explored in PMT research:

- *Cross-version* and *cross-project* settings.
- Models based on *statistical features* (Algorithmic or AL channel) and/or *natural language information* (Natural-Language or NL channel).
- Predictions made at the level of *test suites* or individual *test cases*.

PMTBench is built around two main components:

- An **model** that defines a standard format for representing PMT datasets in a structured and traceable way.
- A **benchmarking framework** for training and evaluating PMT models under different experimental scenarios.

Model for PMT datasets

The PMTBench model defines the main components that are part of a PMT dataset, including the entities involved, the associated properties, and how all elements are connected. It provides a standardized and structured way to describe PMT datasets so that they can be reused, extended, and automatically processed. To represent this information, PMTBench uses the [Resource Description Framework \(RDF\)](#), as its data model, along with the [Turtle](#) syntax, a compact and readable format for writing RDF graphs in plain text.

The model's full structure can be found in the file `standard-pmtbench.ttl`.

The model groups elements into four main categories:

- **Software project:** Describes the basic structure of the system under test using the classes `Project`, `File`, `TestSuite`, and `Test`.
- **Mutations:** Represented by the `Mutant` class, which describes each artificial fault introduced during mutation testing.
- **Collected data:** Covers the information extracted from mutants and tests: `StaticAnalysis`, `Execution`, and `NaturalLanguage`.
- **Tooling metadata:** Contains information about the tools used to generate mutants and collect data: `MutationTool`, `AnalysisTool`, and `CoverageTool`.

Dataset organization

To keep datasets modular and easy to maintain, PMTBench splits them into separate Turtle files:

- **One file per project:** Each TTL file contains all data related to a single project.
- **An index file:** A central TTL file that references all individual project files and serves as the entry point to the dataset.

You can see a practical example of this organization in the folder `ConverterExample`, which includes data for several projects represented in PMTBench's format in different PMT scenarios. In these examples, `index.ttl` serves as the central index file, and the `Projects-TTL` directory contains the individual TTL files for each project.

Framework for Running and Evaluating PMT Models

Converter

The first part of the framework is a converter that takes datasets in CSV format and turns them into Turtle files following the PMTBench's model. This helps reuse existing data in a structured and consistent way. To run the conversion, you need a CSV file listing the projects, plus some extra information, including:

- **Project information:** name, version or commit, repository link, and language.

- **Test suite:** name of the execution, repository link, and optionally target, excluded and added tests.
- **Scenario:** prediction granularity (*ts* for test suite or *tc* for test case) and modeling approach (*AL*, *NL*, or *BOTH*).

In addition, you will need to configure the tools used (mutation, analysis, and coverage tools) directly in the script `project_converter.py`.

The converter covers different scenarios:

- *AL* and *ts*: the CSV files include static and dynamic information at the test-suite level.
- *NL* and *tc*: the CSV files include natural language data for both the mutants and the test cases that exercise them.
- *BOTH* and *tc*: the CSV files include static, dynamic, and natural language data for the mutants and the test cases that execute them.

Example using the data contained in the `ConverterExample` directory, which contains data to illustrate the aforementioned scenarios. Each of the subdirectories includes:

- *projects-common.csv*: a CSV file that defines the set of projects to convert.
- *Projects-CSV*: a directory containing one CSV file per project.
- *index.ttl*: the TTL file that will act as the index for all project-level TTL files.
- *Projects-TTL*: the output directory for the generated TTL files.

In the first scenario (*AL*), the following command processes the projects listed in *projects-common.csv* (`--projects_csv`) and converts the corresponding CSV files in *Projects-CSV* (`--projects_dir`) into TTL files. The output is placed in *Projects-TTL* (`--output_dir`), and *index.ttl* (`--index_file`) is updated accordingly:

```
python3 global_converter.py \
  --projects_dir ConverterExample/AL/Projects-CSV \
  --projects_csv ConverterExample/AL/projects-common.csv \
  --output_dir ConverterExample/AL/Projects-TTL \
  --index_file ConverterExample/AL/index.ttl
```

The converter has been used so far to generate two datasets:

- A new dataset for the *AL* and *ts* scenario, consisting of 200 Java projects. It is intended for use in a *cross-project* setting (citation: `pmtbenchAL`).

- A converted version of the dataset used by Zhao et al., covering the *NL* and *tc* scenario. It includes 6 Java projects, each with multiple versions, and supports *cross-project* and *cross-setting* evaluations (citation: `zhao2024spotting`).

These datasets are publicly available and can be downloaded at: **[URL: to be announced]**.

Filtering

The dataset can be filtered to retrieve only the parts that match certain criteria. You can apply filters based on *project identifier*, *programming language*, *citation*, *mutation tool*, or *type of information* (static, dynamic, and/or natural language data). This makes it easy to adapt the dataset to the needs of a particular study or experiment.

Examples using the data contained in the `FilterExample` directory:

1. Selection of the versions 1 and 5 of the project `'Csv'`, extracting features from the block `naturalLanguage`:

```
python3 main.py \  
  -i FilterExample/index.ttl \  
  -pr Csv_1,Csv_5 \  
  -b naturalLanguage \  
  -o FilterExample/filter1.csv
```

The script will output the file `filter1.csv` with the data of each mutant, and the files `output_Csv_1_test_map.csv` and `output_Csv_1_test_map.csv` with the data of the test cases in each of the versions.

2. Selection of the dataset with citation `'zhao2024spotting'`, extracting features from the block `naturalLanguage`:

```
python3 main.py \  
  -i FilterExample/index.ttl \  
  -b naturalLanguage \  
  -c zhao2024spotting \  
  -o FilterExample/filter2.csv
```

The result will be the same as in the previous example, including also the version 10 of project `Csv`.

3. Selection of mutants in `Java` projects, generated by the mutation tool `PIT_1.15.8`, extracting the features from the blocks `staticAnalysis` and `execution` :

```
python3 main.py \  
  -i FilterExample/index.ttl \  
  -b "staticAnalysis execution" \  
  -t PIT_1.15.8 \  
  -l java \  
  -o FilterExample/filter3.csv
```

Type `python3 main.py --help` for information about the available filters.

Data split

In addition to filtering, the framework also supports splitting the data into training, validation, and test sets.

Traditional partitioning

You can specify the desired proportions using the `--partition` or `-p` option. You can also set the base name of the output files with the `-o` option. The script will automatically append `_training`, `_validation`, and `_test` to this base name to generate the corresponding output files.

Note: the splitting is done at the **project level**, not the mutant level. This means that all instances from the same project will be placed entirely in one of the partitions, either training, validation, or test.

Examples using the data contained in the `DataSplitExample` directory:

1. To split the data with citation `'pmtbenchAL'` into *80% training* and *20% test*, you can run:

```
python3 main.py \  
  -i DataSplitExample/index.ttl \  
  -p 80,20 \  
  -c pmtbenchAL \  
  -b execution staticAnalysis \  
  -o DataSplitExample/split_8020.csv
```

This will generate two files: `split_8020_training.csv` and `split_8020_test.csv`.

2. To split the data with citation `'pmtbenchAL'` into 70% *training*, 15% *validation*, and 15% *test*, use:

```
python3 main.py \  
  -i DataSplitExample/index.ttl \  
  -p 70,15,15 \  
  -c pmtbenchAL \  
  -b execution staticAnalysis \  
  -o DataSplitExample/split_701515.csv
```

In this case, three files will be generated: `split_701515_training.csv`, `split_701515_validation.csv`, and `split_701515_test.csv`.

Partitioning by projects

Instead of specifying percentages, you can explicitly choose which projects go into each partition. This can be useful when you want full control over the training, validation, and test sets. You can do this by using the `--train_projects (-trp)`, `--validation_projects (-vap)`, and `--test_projects (-tep)` options. Note that project names should match the identifiers used in the dataset.

Example using the data contained in the `DataSplitExample` directory:

```
python3 main.py \  
  -i DataSplitExample/index.ttl \  
  -trp Csv_1,Csv_5 \  
  -tep Csv_10 \  
  -b naturalLanguage \  
  -o DataSplitExample/split_Csv.csv
```

With this partition, a model could be trained on versions `Csv_1` and `Csv_5`, and tested on version `Csv_10`. In this case, two files will be generated: `output_training.csv` and `output_test.csv`.

Note: With project-based partitioning, you can easily create a script that runs multiple times with different training and test lists. This makes it straightforward to, for example, have each project serve as the test set once, while training the model on all remaining projects (a setup similar to leave-one-project-out evaluation).

Evaluation of PMT models

To evaluate a PMT model with PMTBench, you will need to provide a script that handles the training and prediction process. For the framework to work with your script, it has to support the following arguments:

- `--training_file` : path to the CSV file with training data.
- `--test_file` : path to the CSV file with test data.
- `--output_file` : path where the script should write its predictions.

The script should generate a CSV file with one row per mutant. Each row should include at least three columns: one for the project, one for the true label, and one for the predicted label. The column names have to be exactly `Project`, `True Label` and `Predicted Label`. Optionally, a fourth column named `Killing Probability` can be included, with a list of predicted probabilities. This is explained in further detail below.

Note: The column `Project` is used to compute the mutation score error per project.

Then, based on the output file containing the predictions, PMTBench reports the following metrics using `sklearn.metrics`:

- Macro precision, recall, and F1-score.
- Weighted precision, recall (accuracy), and F1-score.
- Per-class precision, recall, and F1-score (for the `KILLED` and `SURVIVED` classes).
- Mutation score error per project and on average.

The results can also be exported as:

- A CSV metrics report (option `-mc`).
- A confusion matrix plot (option `-cm`).

The predictions differ depending on the prediction granularity, as it is explained below.

Test-suite level prediction

The model has to assign a single label per mutant representing the result of the whole test suite execution (`KILLED` or `SURVIVED`). Example:

Project	True Label	Predicted Label
Csv_1	KILLED	KILLED
Csv_1	SURVIVED	KILLED
Csv_5	SURVIVED	SURVIVED
Csv_5	KILLED	SURVIVED

Table: Example of a PMT model output for a test file containing two mutants from two different projects. Each project includes one correctly predicted mutant and one mispredicted mutant.

Example using the data contained in the `EvaluationExample` directory. This directory contains a sample model script implementing a decision tree called `decision_tree.py`. You can run it manually with:

```
python3 EvaluationExample/decision_tree.py \
  --training_file EvaluationExample/output_training.csv \
  --test_file EvaluationExample/output_test.csv \
  --output_file EvaluationExample/predicted_labels.csv
```

You can then run it directly through PMTBench by using the `--ml_script` option. The following command filters and splits the data, runs the model, and performs the evaluation:

```
python3 main.py \
  -i EvaluationExample/index.ttl \
  -p 80,20 \
  -c pmtbenchAL \
  -b execution \
  -o EvaluationExample/split_8020.csv \
  --ml_script EvaluationExample/decision_tree.py
```

After running the script, PMTBench uses the generated `split_8020_predicted.csv` file to evaluate the model performance and print test-suite level metrics.

Test-case level prediction

The model has to assign a list of labels (`KILLED` or `SURVIVED`) and, optionally, a list of killing probabilities per mutant, one for each of the test cases covering the mutant.

Example:

Project	True Label	Predicted Label	Killing Probability
Csv_5	['KILLED']	['KILLED']	[0.30]
Csv_5	['KILLED', 'SURVIVED']	['KILLED', 'KILLED']	[0.22, 0.18]
Csv_5	['SURVIVED', 'SURVIVED']	['KILLED', 'SURVIVED']	[0.26, 0.15]

Table: Example of predicted output for three mutants of project Csv_5, with the second and the third one being executed by two test cases. At the test-case level, the model makes 5 predictions in total: 3 correct (KILLED, KILLED) and 2 incorrect (SURVIVED predicted as KILLED).

The test-case level predictions are then aggregated at the test-suite level: if there are no killing probabilities, the mutant is considered KILLED when the predicted label of at least one test case is KILLED; if probabilities exist, the mutant is considered KILLED when at least one test case has a killing probability above a given threshold. Using the example above and a threshold of 0.25 (default threshold), the aggregation output is as follows:

Project	True Label	Predicted Label
Csv_5	KILLED	KILLED: test case with a probability of 0.3 (>0.25)
Csv_5	KILLED	SURVIVED: test cases with a probability of 0.22 and 0.18 (<0.25)
Csv_5	SURVIVED	KILLED: first test case with a probability of 0.26 (>0.25)

Table: At the test-suite level, the model makes 3 predictions in total: 1 correct (KILLED, KILLED) and 2 incorrect (SURVIVED predicted as KILLED or vice versa).

Metrics are reported at both levels, test-case and test-suite level, providing a view of the model's performance on individual test cases and aggregated per mutant. The aggregated results are used for the calculation of the mutation score error. In the case that the aggregated results are based on killing probabilities, a list of thresholds can optionally be provided using the `--thresholds` or `-th` option. Test-suite metrics are then computed separately for each threshold in the list.

Aggregation based on labels

Example using the data contained in the `EvaluationExample` directory. This directory contains a script called `sim_testcase_pred.py`, which simulates the execution of a PMT model in this scenario (it assigns labels randomly, without killing probabilities). You can run it manually with:

```
python3 EvaluationExample/sim_testcase_pred.py \
  --training_file EvaluationExample/output_testcase_training.csv \
  --test_file EvaluationExample/output_testcase_test.csv \
  --output_file EvaluationExample/predicted_testcase_labels.csv
```

You can then run it directly through PMTBench by using the `--ml_script` option. The following command filters and splits the data, runs the model, and performs the evaluation:

```
python3 main.py \
  -i EvaluationExample/index.ttl \
  -trp Csv_1 \
  -tep Csv_5 \
  -b naturalLanguage \
  -o EvaluationExample/split_Csv_1_5.csv \
  --ml_script EvaluationExample/sim_testcase_pred.py
```

After running the script, PMTBench uses the generated `split_Csv_1_5_predicted.csv` file to evaluate model performance and print test-case and test-suite level metrics.

Note: The results will differ from execution to execution, as the predictions are random.

Aggregation based on probabilities

The `EvaluationExample` directory also includes a script called `sim_testcase_pred_prob.py`, which also outputs a list of random killing probabilities. You can then run the evaluation with multiple thresholds as follows:

```
python3 main.py \
  -i EvaluationExample/index.ttl \
  -trp Csv_1 \
  -tep Csv_5 \
  -b naturalLanguage \
  -o EvaluationExample/split_Csv_1_5.csv \
```

```
--ml_script EvaluationExample/sim_testcase_pred_prob.py \  
-mc EvaluationExample/metrics-smp.csv \  
-th 0.1,0.25
```

In this case, two separate CSV files with the metrics will be generated, one for each threshold (0.1 and 0.25), called `metrics-smp_th0.1.csv` and `metrics-smp_th0.25.csv`. Each file contains the test-suite level metrics computed using the corresponding threshold.

Evaluating with existing data

In many PMT experiments, you may want to test a PMT model under different configurations or compare multiple models on the same data. To support this, PMTBench allows you to skip the filtering and splitting steps, and directly use pre-generated training and test files.

This is done using the `--existing_data` option. Here is an example:

```
python3 main.py \  
  --existing_data \  
  -tr EvaluationExample/output_training.csv \  
  -te EvaluationExample/output_test.csv \  
  -o EvaluationExample/output.csv \  
  -m EvaluationExample/decision_tree.py \  
  -cm EvaluationExample/matrix-dt.png \  
  -mc EvaluationExample/metrics-dt.csv
```

This command runs the provided model on the existing data and produces the same evaluation output as before (Test-suite level prediction scenario)

Installation

Requirements

This project requires **Python >= 3.9** and depends on the following Python packages:

- matplotlib==3.8.0
- rdflib==7.1.4
- scikit_learn==1.7.1

- seaborn==0.13.2

Steps

1. Clone this repository or download the source code.
2. (Optional but recommended) Create and activate a virtual environment:

```
python3 -m venv venv  
source venv/bin/activate    # On Linux/Mac  
venv\Scripts\activate       # On Windows
```

3. Install the project dependencies:

```
pip install -r requirements.txt
```